A Project Report

On

“Esports Tournament Tracker and Management System”

ABSTRACT

The Esports Tournament Tracker and Management System is a relational database project designed to organize and manage the core information involved in running esports tournaments, including organizers, games, teams, players, registrations, matches, results, sponsors, and prizes. The system was designed and implemented using Microsoft SQL Server and SQL Server Management Studio (SSMS), following normalized database design principles to eliminate data redundancy and maintain data integrity.

The database schema links related entities through primary and foreign keys, enabling accurate tracking of team registrations, match scheduling, results recording, and dynamic calculation of team rankings and points directly from stored data. Additional SQL Server features such as constraints, joins, subqueries, views, stored procedures, and transactions were used to demonstrate practical database management concepts taught in the course. The resulting system replaces manual, error-prone record-keeping methods with a single, reliable, and query-driven source of tournament information.

TABLE OF CONTATIN

ACKNOWLWDGEMWNT ..... **Error! Bookmark not defined.**

ABSTRACT I

TABLE OF CONTAION II

CHAPTER 1: INTRODUCTION 1

1.1 Project Background 1

1.2 Objectives 2

1.3 Problem Statement 2

1.4 Scope 3

CHAPTER 2: SYSTEM ANALYSIS AND DESIGN 5

2.1 Requirement Analysis 5

Functional Requirements 5

Non-Functional Requirements 5

2.2 Entity Identification 6

2.3 Attributes 7

ORGANIZER 7

GAME 7

TOURNAMENT 7

TEAM 7

PLAYER 7

REGISTRATION 7

MATCH 7

MATCH_RESULT 7

SPONSOR 7

PRIZE 7

2.4 Relationships 8

2.5 ER Diagram 10

2.6 Relational Schema 10
ORGANIZER 10
GAME 11
TOURNAMENT 11
TEAM 11
PLAYER 11
REGISTRATION 11
MATCH 11
MATCH_RESULT 11
SPONSOR 11
PRIZE 11
2.7 Normalization 11
First Normal Form (1NF) 12
Second Normal Form (2NF) 12
Third Normal Form (3NF) 12
CHAPTER 3: DATABASE IMPLEMENTATION 14
3.1 Database Creation 14
3.2 Table Creation Scripts 14
Organizer 14
Game 15
Team 15
Tournament 15
Player 16
Registration 16
Match 16
Match_Result 17
Sponsor 17

Prize 17
3.3 Constraints 18
3.4 Sample Data 20
Organizer 21
Game 21
Team 21
Tournament 21
Player 21
Registration 22
Match 23
Match_Result 23
Sponsor 23
Prize 24
CHAPTER 4: SQL IMPLEMENTATION 26
4.1 INSERT Queries 26
4.2 UPDATE Queries 26
4.3 DELETE Queries 27
4.4 SELECT Queries 28
4.5 WHERE 28
4.6 ORDER BY 29
4.7 Aggregate Functions 30
4.8 GROUP BY 30
4.9 HAVING 31
4.10 JOIN Queries 32
INNER JOIN -Tournament with Organizer 32
INNER JOIN -Tournament with Game 32
INNER JOIN -Teams registered in a specific tournament 32

INNER JOIN -Players with their teams 32
LEFT JOIN -All tournaments, including those without sponsors yet 32
Multiple-table JOIN -Match results with team and match information 33
4.11 Nested Queries 33
Team with the highest total points 33
Tournament with the highest prize pool 34
Players belonging to teams that participated in a particular tournament 34
4.12 Dynamic Leaderboard Query 34
4.13 Views 36
1. TournamentSummaryView 36
2. TeamLeaderboardView 36
3. MatchResultsView 36
4.14 Stored Procedures 37
1. RegisterTeamForTournament 38
2. AddMatchResult 39
3. GetTournamentLeaderboard 39
4.15 Transactions 40
Transaction 1: Register a team into a tournament 41
Transaction 2: Insert a match result safely 41
CHAPTER 5: TESTING AND RESULTS 44
5.1 Testing 44
5.2 Query Outputs and Screenshots 45
5.3 Observations 48
CHAPTER 6: CONCLUSION 50
6.1 Summary 50
6.2 Future Improvements 50
Appendix: Mapping of Implemented Features to DBMS Assignment Requirements 52

TABLE OF FIGURES

Figure 2.1: ER Diagram of the Esports Tournament Tracker.....10

Fig 3.1: EsportsTournametDB Creation 14

Fig 3.2: Tables creation.....18

Fig 3.3:Sample Data Table 25

Fig 4.1:Select Querry.....28

Fig 4.2:Where Querry.....29

Fig 4.3:GROUP BY Querry.....31

Fig 4.4:JOIN Querry.....33

Fig 4.5: Dynamic Leaderboard Query 35

Fig 4.6: View Query 38

Fig 4.7: GetTournamentLeaderboard execution.....40

Fig 4.8: Successful transaction / COMMIT.....42

Fig 4.9: Failed transaction / ROLLBACK.....43

Figure 5.1: Tournament table with sample records 47

Figure 5.2: Dynamic leaderboard query output for Tournament 1.....47

Figure 5.3: Stored procedure GetTournamentLeaderboard execution 47

Figure 5.4: Transaction rollback on duplicate registration.....48

CHAPTER 1: INTRODUCTION

1.1 Project Background

Esports has grown from a small niche hobby into one of the fastest expanding segments of the global entertainment and sports industry. Games such as PUBG Mobile, Mobile Legends: Bang Bang, Valorant, Dota 2, and Free Fire now host tournaments with hundreds of participating teams, thousands of individual players, large prize pools, and multiple corporate sponsors. As the number of tournaments increases, organizers are required to manage an increasing volume of interconnected information, including organizer details, participating teams, individual players, match schedules, match results, sponsorships, and prize distribution.

When this information is managed manually, using spreadsheets, paper records, or informal documents, several problems typically arise. Team and player records are duplicated across different tournaments, match schedules become difficult to track, points and rankings are calculated by hand and are prone to error, and there is no single reliable source of information that connects a tournament to its organizer, teams, matches, results, sponsors, and prizes. As the number of tournaments and participants grows, this manual approach becomes increasingly unreliable and time-consuming.

A relational database management system (RDBMS) is well suited to solve these problems because it allows related pieces of information organizers, games, tournaments, teams, players, registrations, matches, results, sponsors, and prizes to be stored in separate, well-structured tables that are linked together through primary and foreign keys. This eliminates redundant data, enforces data integrity through constraints, and allows complex information (such as a tournament leaderboard) to be generated on demand using SQL queries rather than being stored and manually maintained.

The purpose of this project, the Esports Tournament Tracker and Management System, is to design and implement a normalized relational database using Microsoft SQL Server and SQL Server Management Studio (SSMS) that can store and manage all core information related to running esports tournaments, and to demonstrate the practical application of database design principles and SQL Server features taught in the DBMS course.

1.2 Objectives

The objectives of this project are as follows:

- To design a normalized relational database for esports tournament management.
- To manage information about games supported by the system.
- To maintain accurate records of tournament organizers.
- To manage information about participating teams and their players.
- To handle the registration of teams into tournaments.
- To schedule and manage matches within a tournament.
- To record and store the results of each match.
- To calculate team rankings and points dynamically from stored match results.
- To manage information about tournament sponsors and prize distribution.
- To demonstrate core SQL Server DBMS concepts, including constraints, joins, subqueries, views, stored procedures, and transactions.

1.3 Problem Statement

Esports tournaments that are organized without a centralized database system commonly face the following problems:

- Difficulty maintaining accurate and up-to-date team and player information across multiple tournaments.
- Duplicate or inconsistent records for the same team or player.
- Difficulty tracking which teams have registered for which tournaments, and whether that registration has been confirmed.
- Difficulty managing match schedules, including match dates, stages, and maps.
- Errors in manually calculating placement points, kill points, and total points for each team.
- Difficulty generating accurate, up-to-date rankings or leaderboards for a tournament.
- Difficulty maintaining organized records of sponsors and the prize amounts awarded to each team.

- Absence of a single, centralized, and reliable source of information connecting organizers, games, tournaments, teams, players, matches, results, sponsors, and prizes.

The Esports Tournament Tracker and Management System addresses these problems by organizing all related information into normalized relational tables connected through primary and foreign keys. Constraints such as NOT NULL, UNIQUE, CHECK, and DEFAULT ensure that only valid and consistent data can be entered. Match results are stored at their most granular level (per team, per match), and the tournament leaderboard is generated dynamically through SQL aggregate queries rather than being stored and manually recalculated, removing the possibility of the leaderboard becoming out of sync with the underlying match data.

1.4 Scope

The Esports Tournament Tracker and Management System, as implemented in this project, covers the following areas:

- Game management - storing information about the esports games supported by the system.
- Tournament management - storing tournament details, schedule, prize pool, and status.
- Organizer management - storing information about the individuals or organizations running tournaments.
- Team management - storing information about participating teams.
- Player management - storing information about individual players and their team affiliation.
- Tournament registration - recording which teams have registered for which tournaments.
- Match scheduling - recording matches conducted as part of a tournament.
- Match result management - recording each team's performance in each match.
- Ranking and leaderboard generation - dynamically calculating team standings from match results.
- Sponsor management - recording sponsors associated with a tournament.

- Prize management - recording prize amounts and the teams that receive them.

This project focuses on the database design and SQL Server implementation of the system, including table design, constraints, sample data, queries, views, stored procedures, and transactions. It does not include the development of a complete public-facing website, mobile application, or graphical user interface; such components are identified as future improvements in Chapter 6. The system is designed and tested using Microsoft SQL Server Management Studio (SSMS).

CHAPTER 2: SYSTEM ANALYSIS AND DESIGN

2.1 Requirement Analysis

Functional Requirements

The system shall allow the following operations:

- Add and manage games supported by the system.
- Add and manage organizer information.
- Create and manage tournaments linked to a game and an organizer.
- Register teams for tournaments.
- Add and manage players and associate them with teams.
- Schedule and manage matches for a tournament.
- Record match results for each team in a match.
- Calculate team points and rankings from stored match results.
- Generate a dynamic leaderboard for each tournament.
- Manage sponsor information for tournaments.
- Manage prize information for tournaments.
- Retrieve summarized tournament reports using views, joins, and aggregate queries.

Non-Functional Requirements

- Data Integrity - Primary key, foreign key, CHECK, and UNIQUE constraints ensure that only valid and consistent data is stored.
- Reliability - Transactions with COMMIT and ROLLBACK ensure that related operations either fully succeed or fully fail.
- Security - Access to the database can be restricted through SQL Server logins, users, and permissions.
- Maintainability - A normalized schema (up to 3NF) minimizes redundancy and simplifies future changes.

- Scalability - The schema can accommodate an increasing number of tournaments, teams, players, and matches without structural changes.
- Performance - Primary and foreign key columns are indexed by default, which speeds up joins and lookups.
- Usability - Views and stored procedures simplify common operations for end users and applications.

2.2 Entity Identification

Based on the requirements above, ten core entities were identified for the Esports Tournament Tracker and Management System. Each entity represents a distinct real-world object or association that needs to be independently tracked in the database.

Entity	Purpose
Organizer	Stores information about the person/organization responsible for a tournament.
Game	Stores information about the esports games supported by the system.
Tournament	Stores details of an individual esports tournament.
Team	Stores information about participating esports teams.
Player	Stores information about individual players and their team affiliation.
Registration	Associative entity recording which team registered for which tournament.
Match	Stores matches conducted as part of a tournament.
Match_Result	Associative entity recording a team's performance in a specific match.
Sponsor	Stores information about sponsors associated with a tournament.
Prize	Stores prize information and the team that receives a tournament position.

2.3 Attributes

The attributes of each entity, with primary keys (PK) and foreign keys (FK) identified, are listed below.

ORGANIZER

OrganizerID (PK), Name, Email (UNIQUE), Phone, Organization

GAME

GameID (PK), GameName (UNIQUE), Genre, TeamSize

TOURNAMENT

TournamentID (PK), TournamentName, StartDate, EndDate, PrizePool, Status, OrganizerID (FK), GameID (FK)

TEAM

TeamID (PK), TeamName (UNIQUE), CaptainName, Region, CoachName

PLAYER

PlayerID (PK), PlayerName, IGN (UNIQUE), Role, Country, TeamID (FK)

REGISTRATION

RegistrationID (PK), RegistrationDate, Status, TournamentID (FK), TeamID (FK)

MATCH

MatchID (PK), MatchNumber, MatchDate, MapName, Stage, TournamentID (FK)

MATCH_RESULT

ResultID (PK), Placement, KillPoints, PlacementPoints, TotalPoints (derived), MatchID (FK), TeamID (FK)

SPONSOR

SponsorID (PK), SponsorName, Amount, TournamentID (FK)

PRIZE

PrizeID (PK), Position, PrizeAmount, TournamentID (FK), TeamID (FK)

2.4 Relationships

The following relationships exist between the entities of the Esports Tournament Tracker and Management System:

#	Relationship	Cardinality	Description
1	Organizer → Tournament	1 : M	One organizer can organize many tournaments; each tournament has exactly one organizer.
2	Game → Tournament	1 : M	One game can be featured in many tournaments; each tournament features exactly one game.
3	Tournament → Match	1 : M	One tournament consists of many matches; each match belongs to exactly one tournament.
4	Tournament → Registration	1 : M	One tournament can have many registrations; each registration belongs to one tournament.
5	Team → Registration	1 : M	One team can have many registrations (for different tournaments); each registration belongs to one team.

#	Relationship	Cardinality	Description
6	Team → Player	1 : M	One team has many players; each player belongs to exactly one team.
7	Match → Match_Result	1 : M	One match produces many match results (one per participating team); each result belongs to one match.
8	Team → Match_Result	1 : M	One team can have many match results (across different matches); each result belongs to one team.
9	Tournament → Sponsor	1 : M	One tournament can have many sponsors; each sponsor record belongs to one tournament.
10	Tournament → Prize	1 : M	One tournament can have many prize entries (one per position); each prize belongs to one tournament.
11	Team → Prize	1 : M	One team can win prizes in different tournaments; each prize entry is awarded to one team.

Two of these relationships deserve special explanation because they resolve underlying many-to-many relationships:

Registration: A tournament can have many teams, and a team can participate in many tournaments. This is a many-to-many relationship between Tournament and Team. The Registration entity resolves this many-to-many relationship by acting as an associative (junction) entity: each row in Registration links exactly one TournamentID with exactly one TeamID, along with the registration date and status. In relational terms, Tournament and Team both have a 1:M relationship with Registration.

Match_Result: A match can involve many teams, and a team can participate in many matches. This is a many-to-many relationship between Match and Team. The Match_Result entity resolves this many-to-many relationship by acting as an associative entity: each row in Match_Result links exactly one MatchID with exactly

one TeamID, along with that team's placement, kill points, and total points for that specific match. Match and Team both have a 1:M relationship with Match_Result.

2.5 ER Diagram

The Entity-Relationship diagram below is presented using Crow's Foot notation. Each box represents an entity with its attributes; the primary key (PK) is underlined/marked and foreign keys (FK) are marked accordingly. A single tick mark near an entity represents the “one” side of a relationship, and a crow's-foot (three-pronged) mark represents the “many” side.

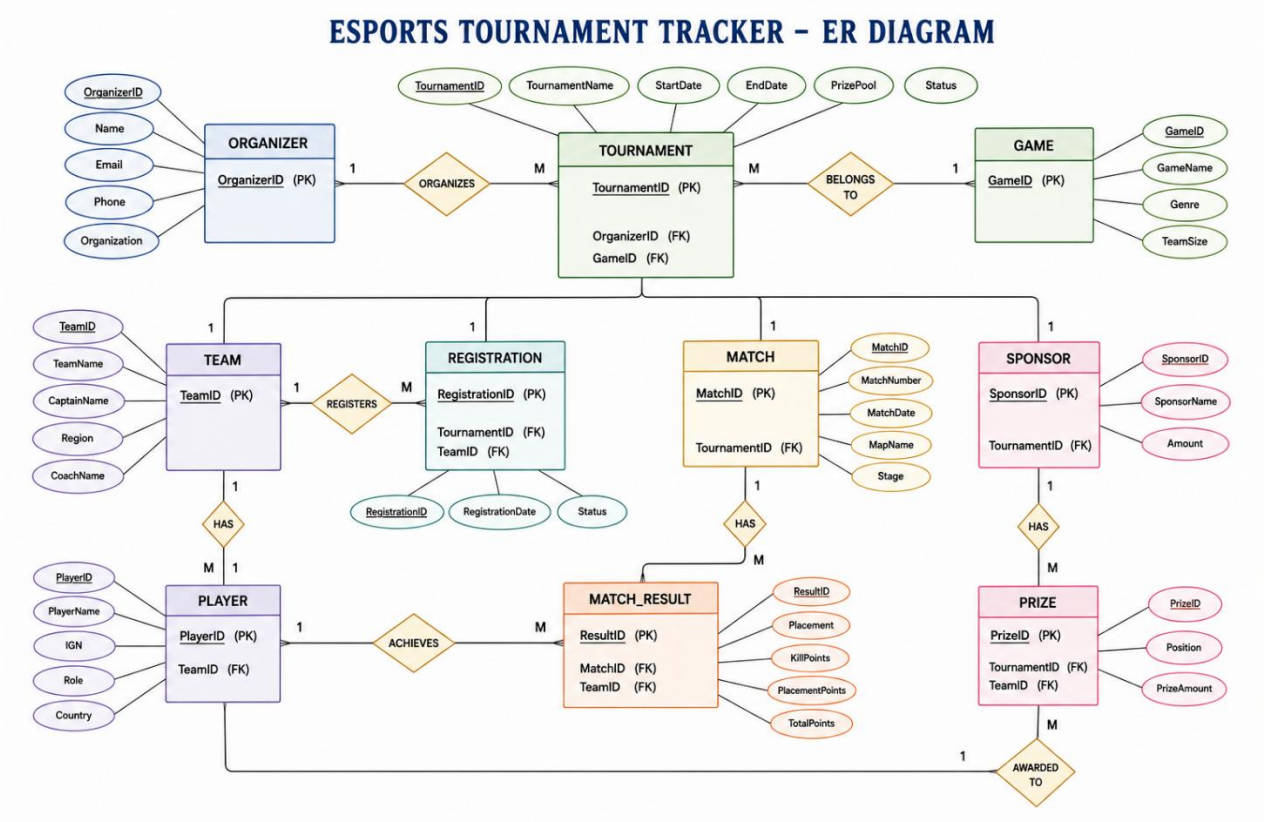


Figure 2.1: ER Diagram of the Esports Tournament Tracker

2.6 Relational Schema

The ER model above is converted into the following relational schema. Primary keys are shown in bold and underlined style notation (PK), and foreign keys are marked (FK).

ORGANIZER

```
Organizer(OrganizerID PK, Name, Email UNIQUE, Phone, Organization)
```

GAME

```
Game(GameID PK, GameName UNIQUE, Genre, TeamSize)
```

TOURNAMENT

```
Tournament(TournamentID PK, TournamentName, StartDate, EndDate, PrizePool, Status,
OrganizerID FK → Organizer.OrganizerID, GameID FK → Game.GameID)
```

TEAM

```
Team(TeamID PK, TeamName UNIQUE, CaptainName, Region, CoachName)
```

PLAYER

```
Player(PlayerID PK, PlayerName, IGN UNIQUE, Role, Country, TeamID FK → Team.TeamID)
```

REGISTRATION

```
Registration(RegistrationID PK, RegistrationDate, Status, TournamentID FK →
Tournament.TournamentID, TeamID FK → Team.TeamID) -UNIQUE(TournamentID, TeamID)
```

MATCH

```
Match(MatchID PK, MatchNumber, MatchDate, MapName, Stage, TournamentID FK →
Tournament.TournamentID) -UNIQUE(TournamentID, MatchNumber)
```

MATCH_RESULT

```
Match_Result(ResultID PK, Placement, KillPoints, PlacementPoints, TotalPoints,
MatchID FK → Match.MatchID, TeamID FK → Team.TeamID) -UNIQUE(MatchID, TeamID)
```

SPONSOR

```
Sponsor(SponsorID PK, SponsorName, Amount, TournamentID FK →
Tournament.TournamentID)
```

PRIZE

```
Prize(PrizeID PK, Position, PrizeAmount, TournamentID FK → Tournament.TournamentID,
TeamID FK → Team.TeamID) -UNIQUE(TournamentID, Position)
```

2.7 Normalization

The relational schema was designed by applying normalization principles up to the Third Normal Form (3NF) in order to minimize data redundancy and avoid update, insertion, and deletion anomalies.

First Normal Form (1NF)

A relation is in 1NF if all attribute values are atomic (indivisible) and there are no repeating groups or multi-valued attributes. In the initial design, an entity such as Team could have been modeled with a repeating group of players (e.g. Player1, Player2, Player3, ... in the same row), which would violate 1NF. This was avoided by creating a separate Player entity, where each player is stored as a single row with a single, atomic set of attributes (PlayerName, IGN, Role, Country) and a foreign key TeamID linking back to the owning team. Similarly, Match_Result stores exactly one placement, one kill-point value, and one placement-point value per team per match, rather than combining multiple teams' results into one row. All attributes in every table of this design (e.g. Email, Phone, TournamentName, MapName) hold single, atomic values, so every table in the schema satisfies 1NF.

Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the whole primary key, not just part of it. This rule only applies to tables with a composite primary key. In this design, every table uses a single-column surrogate primary key (an auto-incrementing ID such as TournamentID or MatchID), so there is no composite primary key and therefore no possibility of a partial dependency. For example, in Registration, the primary key is the single column RegistrationID (not the combination of TournamentID and TeamID); RegistrationDate and Status depend fully on RegistrationID. The same reasoning applies to Match_Result, where Placement, KillPoints, and PlacementPoints depend fully on ResultID. Because no non-key attribute depends on only part of a composite key, all tables in this schema satisfy 2NF.

Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and it has no transitive dependency, meaning no non-key attribute depends on another non-key attribute rather than depending directly on the primary key. This principle was applied while designing the Tournament table: attributes such as OrganizerName or OrganizerEmail were deliberately not stored

inside the Tournament table, because they depend on OrganizerID (a non-key attribute of Tournament, but the key of Organizer) rather than directly on TournamentID. Instead, Tournament stores only the foreign key OrganizerID, and organizer details are retrieved through a JOIN with the Organizer table whenever needed. The same reasoning was applied to Game details in Tournament, Team details in Player, Registration, and Match_Result, and Tournament details in Match, Sponsor, and Prize. The most important application of 3NF in this project is the exclusion of a separate, stored Leaderboard table: a team's total points and ranking are transitively dependent on the individual match results (KillPoints and PlacementPoints in Match_Result) rather than being an independent fact about a tournament or team. Storing a leaderboard as its own table would duplicate derived data and risk it becoming inconsistent with the underlying Match_Result rows. Instead, the leaderboard is always calculated dynamically using SQL aggregate functions (SUM, GROUP BY) over Match_Result, as shown in Chapter 4. Because every non-key attribute in every table depends only on that table's primary key, all ten tables in this schema satisfy 3NF.

In summary, the final schema -Organizer, Game, Tournament, Team, Player, Registration, Match, Match_Result, Sponsor, and Prize is fully normalized to 3NF: attributes are atomic (1NF), non-key attributes depend on the whole of a single-column primary key (2NF), and no non-key attribute depends transitively on another non-key attribute (3NF).

CHAPTER 3: DATABASE IMPLEMENTATION

3.1 Database Creation

The database was created in Microsoft SQL Server Management Studio (SSMS) using the following script:

```
CREATE DATABASE EsportsTournamentDB;
GO

USE EsportsTournamentDB;
GO
```

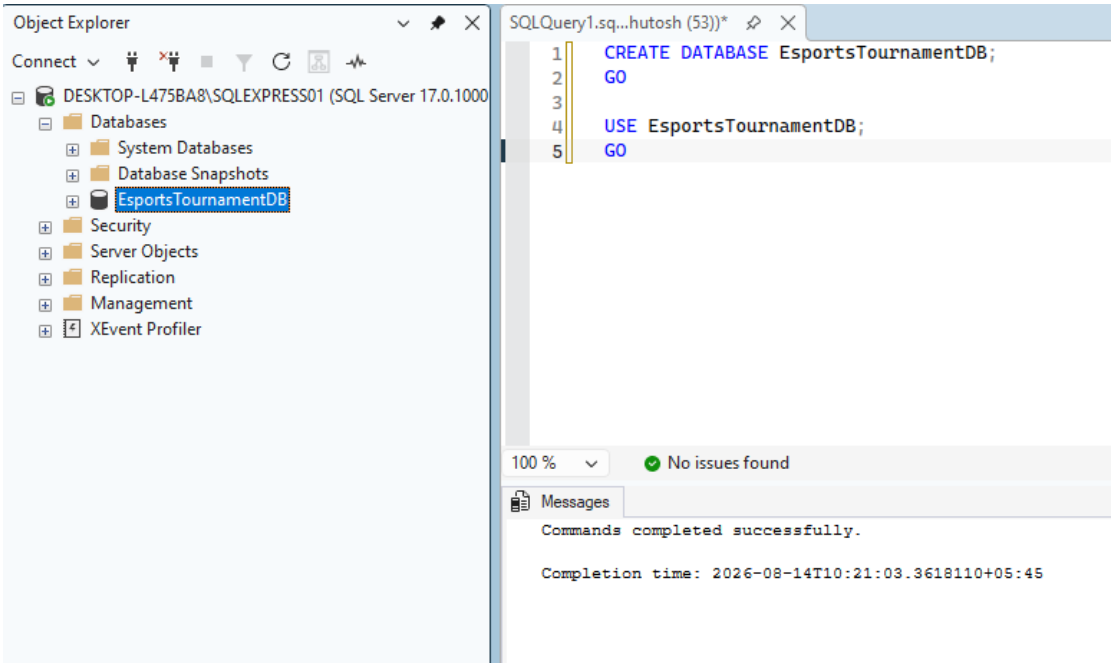


Fig 3.1: EsportsTournametDB Creation

3.2 Table Creation Scripts

The tables are created in an order that respects foreign-key dependencies: independent tables (Organizer, Game, Team) are created first, followed by tables that reference them (Tournament, Player), and finally the associative and dependent tables (Registration, Match, Match_Result, Sponsor, Prize).

Organizer

```
CREATE TABLE Organizer (
    OrganizerID   INT IDENTITY(1,1) PRIMARY KEY,
```

```
    Name          VARCHAR(100) NOT NULL,
    Email         VARCHAR(100) NOT NULL UNIQUE,
    Phone         VARCHAR(15)  NOT NULL,
    Organization  VARCHAR(100) NULL
);
```

Game

```
CREATE TABLE Game (
    GameID    INT IDENTITY(1,1) PRIMARY KEY,
    GameName  VARCHAR(50) NOT NULL UNIQUE,
    Genre     VARCHAR(50) NOT NULL,
    TeamSize  INT NOT NULL CHECK (TeamSize > 0)
);
```

Team

```
CREATE TABLE Team (
    TeamID        INT IDENTITY(1,1) PRIMARY KEY,
    TeamName      VARCHAR(100) NOT NULL UNIQUE,
    CaptainName   VARCHAR(100) NOT NULL,
    Region        VARCHAR(50)  NOT NULL,
    CoachName     VARCHAR(100) NULL
);
```

Tournament

```
CREATE TABLE Tournament (
    TournamentID    INT IDENTITY(1,1) PRIMARY KEY,
    TournamentName  VARCHAR(100) NOT NULL,
    StartDate       DATE NOT NULL,
    EndDate         DATE NOT NULL,
    PrizePool       DECIMAL(12,2) NOT NULL DEFAULT 0,
    Status          VARCHAR(20) NOT NULL DEFAULT 'Upcoming',
    OrganizerID     INT NOT NULL,
    GameID          INT NOT NULL,
    CONSTRAINT FK_Tournament_Organizer FOREIGN KEY (OrganizerID)
        REFERENCES Organizer(OrganizerID),
    CONSTRAINT FK_Tournament_Game FOREIGN KEY (GameID)
        REFERENCES Game(GameID),
    CONSTRAINT CK_Tournament_Dates CHECK (EndDate >= StartDate),
    CONSTRAINT CK_Tournament_PrizePool CHECK (PrizePool >= 0),
    CONSTRAINT CK_Tournament_Status CHECK (Status IN
        ('Upcoming','Ongoing','Completed','Cancelled'))
);
```

Player

```
CREATE TABLE Player (
    PlayerID    INT IDENTITY(1,1) PRIMARY KEY,
    PlayerName  VARCHAR(100) NOT NULL,
    IGN         VARCHAR(50)  NOT NULL UNIQUE,
    Role        VARCHAR(30)  NOT NULL,
    Country     VARCHAR(50)  NOT NULL,
    TeamID      INT NOT NULL,
    CONSTRAINT FK_Player_Team FOREIGN KEY (TeamID)
        REFERENCES Team(TeamID)
);
```

Registration

```
CREATE TABLE Registration (
    RegistrationID    INT IDENTITY(1,1) PRIMARY KEY,
    RegistrationDate  DATE NOT NULL DEFAULT GETDATE(),
    Status            VARCHAR(20) NOT NULL DEFAULT 'Pending',
    TournamentID      INT NOT NULL,
    TeamID            INT NOT NULL,
    CONSTRAINT FK_Registration_Tournament FOREIGN KEY (TournamentID)
        REFERENCES Tournament(TournamentID),
    CONSTRAINT FK_Registration_Team FOREIGN KEY (TeamID)
        REFERENCES Team(TeamID),
    CONSTRAINT CK_Registration_Status CHECK (Status IN
        ('Pending','Confirmed','Cancelled')),
    CONSTRAINT UQ_Registration_Tournament_Team UNIQUE (TournamentID, TeamID)
);
```

Match

```
CREATE TABLE Match (
    MatchID       INT IDENTITY(1,1) PRIMARY KEY,
    MatchNumber   INT NOT NULL,
    MatchDate     DATE NOT NULL,
    MapName       VARCHAR(50) NOT NULL,
    Stage         VARCHAR(30) NOT NULL,
    TournamentID  INT NOT NULL,
    CONSTRAINT FK_Match_Tournament FOREIGN KEY (TournamentID)
        REFERENCES Tournament(TournamentID),
    CONSTRAINT CK_Match_Stage CHECK (Stage IN
        ('Group Stage','Quarterfinal','Semifinal','Final')),
    CONSTRAINT UQ_Match_Tournament_Number UNIQUE (TournamentID, MatchNumber)
);
```

Match_Result

TotalPoints is implemented as a SQL Server computed column so that it is always automatically and consistently derived from KillPoints and PlacementPoints, exactly as required ("TotalPoints should logically represent the points earned from placement and kills"), without allowing the application to insert an inconsistent value.

```
CREATE TABLE Match_Result (
    ResultID          INT IDENTITY(1,1) PRIMARY KEY,
    Placement         INT NOT NULL CHECK (Placement > 0),
    KillPoints        INT NOT NULL DEFAULT 0 CHECK (KillPoints >= 0),
    PlacementPoints   INT NOT NULL DEFAULT 0 CHECK (PlacementPoints >= 0),
    TotalPoints        AS (KillPoints + PlacementPoints) PERSISTED,
    MatchID           INT NOT NULL,
    TeamID            INT NOT NULL,
    CONSTRAINT FK_Result_Match FOREIGN KEY (MatchID)
        REFERENCES Match(MatchID),
    CONSTRAINT FK_Result_Team FOREIGN KEY (TeamID)
        REFERENCES Team(TeamID),
    CONSTRAINT UQ_Result_Match_Team UNIQUE (MatchID, TeamID)
);
```

Sponsor

```
CREATE TABLE Sponsor (
    SponsorID     INT IDENTITY(1,1) PRIMARY KEY,
    SponsorName   VARCHAR(100) NOT NULL,
    Amount        DECIMAL(12,2) NOT NULL CHECK (Amount >= 0),
    TournamentID  INT NOT NULL,
    CONSTRAINT FK_Sponsor_Tournament FOREIGN KEY (TournamentID)
        REFERENCES Tournament(TournamentID)
);
```

Prize

```
CREATE TABLE Prize (
    PrizeID       INT IDENTITY(1,1) PRIMARY KEY,
    Position      INT NOT NULL CHECK (Position > 0),
    PrizeAmount   DECIMAL(12,2) NOT NULL CHECK (PrizeAmount >= 0),
    TournamentID  INT NOT NULL,
    TeamID        INT NOT NULL,
    CONSTRAINT FK_Prize_Tournament FOREIGN KEY (TournamentID)
        REFERENCES Tournament(TournamentID),
    CONSTRAINT FK_Prize_Team FOREIGN KEY (TeamID)
```

```
        REFERENCES Team(TeamID),
    CONSTRAINT UQ_Prize_Tournament_Position UNIQUE (TournamentID, Position)
);
```

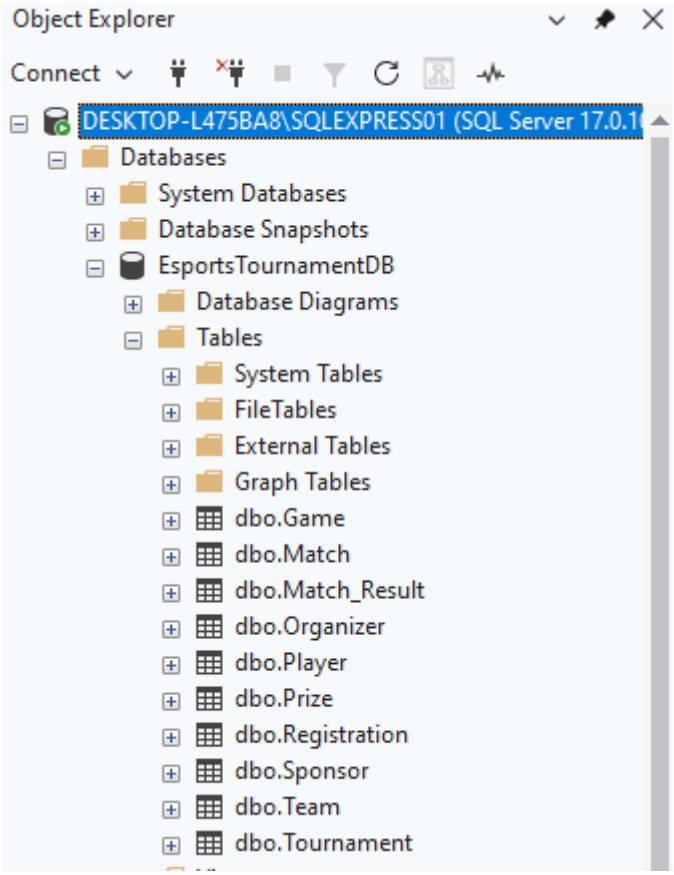


Fig 3.2: Tables creation

3.3 Constraints

The table creation scripts above use the following categories of constraints to enforce data integrity:

Constraint Type	Example in This Project	Purpose
PRIMARY KEY	OrganizerID, GameID, TournamentID, ...	Uniquely identifies each row in a table.
FOREIGN KEY	Tournament.OrganizerID → Organizer.OrganizerID	Enforces referential integrity between related tables.

Constraint Type	Example in This Project	Purpose
NOT NULL	Organizer.Name, Tournament.StartDate	Ensures essential data is always provided.
UNIQUE	Organizer.Email, Game.GameName, Team.TeamName, Player.IGN	Prevents duplicate values for attributes that must be distinct.
CHECK	Game.TeamSize > 0, Tournament.PrizePool >= 0, Match_Result.Placement > 0	Restricts a column to a valid range or set of values.
DEFAULT	Tournament.Status DEFAULT 'Upcoming', Registration.RegistrationDate DEFAULT GETDATE()	Supplies a sensible value automatically when none is provided.
Composite UNIQUE	UQ_Registration_Tournament_Team (TournamentID, TeamID)	Prevents the same team from registering twice for the same tournament.
Computed Column	Match_Result.TotalPoints = KillPoints + PlacementPoints	Automatically keeps TotalPoints consistent with its source values.

A few of these constraints are particularly important and are explained below:

- Unique email for Organizer (Organizer.Email UNIQUE) -prevents two organizer records from sharing the same email address.
- Unique game name (Game.GameName UNIQUE) -prevents the same game from being entered twice under different IDs.
- Valid tournament dates (CK_Tournament_Dates: EndDate >= StartDate) - prevents logically impossible tournament schedules.
- PrizePool >= 0 and Amount/PrizeAmount >= 0 -prevents negative monetary values.
- TeamSize > 0 -ensures a game always requires at least one player per team.

- Placement > 0 and Position > 0 -ensures placement/position values are meaningful (1st, 2nd, 3rd, ...).
- KillPoints >= 0 and PlacementPoints >= 0 -prevents negative point values.
- Registration Status restricted using CHECK to 'Pending', 'Confirmed', or 'Cancelled'.
- Tournament Status restricted using CHECK to 'Upcoming', 'Ongoing', 'Completed', or 'Cancelled'.
- Default registration status of 'Pending' and a default RegistrationDate of the current date, applied automatically when not explicitly supplied.

3.4 Sample Data

Realistic sample data was inserted into each table to support the demonstration of SQL queries in Chapter 4. A summary of the sample data volume is given below; the full INSERT statements are provided in Section 4.1.

Table	Approx. Records
Organizer	4
Game	5
Tournament	5
Team	8
Player	24
Registration	14
Match	12
Match_Result	32
Sponsor	8
Prize	7

Realistic esports titles such as PUBG Mobile, Mobile Legends: Bang Bang, Valorant, Free Fire, and Dota 2 are used as sample games, and fictional but realistic team names,

player IGNs, and tournament names are used throughout the sample data. The complete data-loading scripts, in foreign-key-safe order, are given below.

Organizer

```
INSERT INTO Organizer (Name, Email, Phone, Organization) VALUES
('Rajesh Sharma', 'rajesh.sharma@esportsnp.com', '9841000001', 'ProGaming Nepal'),
('Anita Gurung', 'anita.gurung@levelupevents.com', '9841000002', 'LevelUp Events'),
('Bishal Karki', 'bishal.karki@arenaesports.com', '9841000003', 'Arena Esports'),
('Sunita Rai', 'sunita.rai@championsleague.com', '9841000004', 'Champions League
Esports');
```

Game

```
INSERT INTO Game (GameName, Genre, TeamSize) VALUES
('PUBG Mobile', 'Battle Royale', 4),
('Mobile Legends: Bang Bang', 'MOBA', 5),
('Valorant', 'Tactical Shooter', 5),
('Free Fire', 'Battle Royale', 4),
('Dota 2', 'MOBA', 5);
```

Team

```
INSERT INTO Team (TeamName, CaptainName, Region, CoachName) VALUES
('Nepal Tigers', 'Suman Thapa', 'Kathmandu', 'Prakash Bhandari'),
('Himalayan Wolves', 'Kiran Magar', 'Pokhara', 'Rita Shrestha'),
('Kathmandu Kings', 'Bikash Lama', 'Kathmandu', 'Deepak Adhikari'),
('Everest Esports', 'Sabin Tamang', 'Kathmandu', NULL),
('Pokhara Panthers', 'Nabin Gurung', 'Pokhara', 'Sarita KC'),
('Lumbini Legends', 'Rajan Chaudhary', 'Lumbini', NULL),
('Chitwan Chargers', 'Anish Bhatt', 'Chitwan', 'Manisha Rana'),
('Dharan Dragons', 'Sujit Rai', 'Dharan', NULL);
```

Tournament

```
INSERT INTO Tournament (TournamentName, StartDate, EndDate, PrizePool, Status,
OrganizerID, GameID) VALUES
('PMGC Nepal Series 2026', '2026-01-10', '2026-01-20', 500000, 'Completed', 1, 1),
('MLBB Champions Cup', '2026-02-01', '2026-02-10', 300000, 'Ongoing', 2, 2),
('Valorant Masters Kathmandu', '2026-03-05', '2026-03-15', 700000, 'Upcoming', 3,
3),
('Free Fire Fury League', '2026-01-25', '2026-01-30', 200000, 'Completed', 1, 4),
('Dota 2 Himalayan Cup', '2026-04-01', '2026-04-12', 450000, 'Upcoming', 4, 5);
```

Player

```
INSERT INTO Player (PlayerName, IGN, Role, Country, TeamID) VALUES
('Suman Thapa','SumanX','IGL','Nepal',1),
('Aakash Basnet','AakashSniper','Assaulter','Nepal',1),
('Ramesh KC','RameshRuler','Support','Nepal',1),
('Kiran Magar','KiranWolf','IGL','Nepal',2),
('Bibek Rana','BibekBlaze','Fragger','Nepal',2),
('Manish Thapa','ManishM','Support','Nepal',2),
('Bikash Lama','BikashKing','IGL','Nepal',3),
('Suraj Shah','SurajFire','Fragger','Nepal',3),
('Nirajan Pandey','NirajanN','Support','Nepal',3),
('Sabin Tamang','SabinPeak','IGL','Nepal',4),
('Yubraj Karki','YubrajY','Fragger','Nepal',4),
('Prashant Poudel','PrashantP','Support','Nepal',4),
('Nabin Gurung','NabinP','IGL','Nepal',5),
('Milan Gharti','MilanG','Fragger','Nepal',5),
('Sandesh Rana','SandeshR','Support','Nepal',5),
('Rajan Chaudhary','RajanL','IGL','Nepal',6),
('Bimal Yadav','BimalY','Fragger','Nepal',6),
('Sanjay Mahato','SanjayM','Support','Nepal',6),
('Anish Bhatt','AnishC','IGL','Nepal',7),
('Roshan Ghimire','RoshanG','Fragger','Nepal',7),
('Deepika Thapa','DeepikaT','Support','Nepal',7),
('Sujit Rai','SujitD','IGL','Nepal',8),
('Alina Limbu','AlinaL','Fragger','Nepal',8),
('Bishwas Rai','BishwasR','Support','Nepal',8);
```

Registration

```
INSERT INTO Registration (RegistrationDate, Status, TournamentID, TeamID) VALUES
('2025-12-20','Confirmed',1,1),
('2025-12-20','Confirmed',1,2),
('2025-12-21','Confirmed',1,3),
('2025-12-21','Confirmed',1,4),
('2026-01-15','Confirmed',2,3),
('2026-01-15','Confirmed',2,4),
('2026-01-16','Confirmed',2,6),
('2026-01-17','Pending',2,7),
('2026-02-15','Confirmed',3,1),
('2026-02-15','Confirmed',3,2),
('2026-02-16','Confirmed',3,8),
('2025-12-28','Confirmed',4,5),
('2025-12-28','Confirmed',4,6),
('2025-12-29','Confirmed',4,7);
```

Match

MatchIDs are generated automatically in insertion order (1–12): matches 1–4 belong to Tournament 1, 5–8 to Tournament 2, and 9–12 to Tournament 4.

```
INSERT INTO Match (MatchNumber, MatchDate, MapName, Stage, TournamentID) VALUES
(1,'2026-01-10','Erangel','Group Stage',1),
(2,'2026-01-12','Miramar','Group Stage',1),
(3,'2026-01-16','Sanhok','Semifinal',1),
(4,'2026-01-20','Erangel','Final',1),
(1,'2026-02-01','Land of Dawn','Group Stage',2),
(2,'2026-02-03','Land of Dawn','Group Stage',2),
(3,'2026-02-06','Land of Dawn','Semifinal',2),
(4,'2026-02-10','Land of Dawn','Final',2),
(1,'2026-01-25','Bermuda','Group Stage',4),
(2,'2026-01-27','Purgatory','Group Stage',4),
(3,'2026-01-29','Bermuda','Semifinal',4),
(4,'2026-01-30','Bermuda','Final',4);
```

Match_Result

TotalPoints is a computed column and is therefore omitted from the INSERT column list; SQL Server calculates it automatically as KillPoints + PlacementPoints.

```
INSERT INTO Match_Result (Placement, KillPoints, PlacementPoints, MatchID, TeamID)
VALUES
(1,10,10,1,1), (2,6,8,1,2), (3,4,6,1,3), (4,2,4,1,4),
(1,12,10,2,2), (2,8,8,2,1), (3,5,6,2,4), (4,3,4,2,3),
(1,9,10,3,1), (2,7,8,3,2),
(1,11,10,4,1), (2,9,8,4,2),
(1,5,10,5,3), (2,3,8,5,4),
(1,4,10,6,6), (2,2,8,6,7),
(1,6,10,7,3), (2,3,8,7,6),
(1,7,10,8,3), (2,4,8,8,7),
(1,10,10,9,5), (2,7,8,9,6), (3,4,6,9,7), (4,2,4,9,8),
(1,9,10,10,6), (2,8,8,10,5), (3,5,6,10,8), (4,3,4,10,7),
(1,6,10,11,5), (2,4,8,11,6),
(1,8,10,12,5), (2,5,8,12,6);
```

Sponsor

```
INSERT INTO Sponsor (SponsorName, Amount, TournamentID) VALUES
('TitanTech Gadgets', 150000, 1),
('RedBull Energy', 100000, 1),
('NexaMobile', 80000, 2),
```

```
('GameFuel Drinks', 60000, 2),
('ProGear Peripherals', 120000, 3),
('CyberNet ISP', 90000, 3),
('Everest Bank', 70000, 4),
('Himalayan Java Coffee', 50000, 4);
```

Prize

Prizes are recorded for the two completed tournaments (Tournament 1 and Tournament 4); the ongoing and upcoming tournaments do not yet have final standings.

```
INSERT INTO Prize (Position, PrizeAmount, TournamentID, TeamID) VALUES
(1, 300000, 1, 1),
(2, 150000, 1, 2),
(3, 50000, 1, 4),
(4, 20000, 1, 3),
(1, 100000, 4, 5),
```

```
(2, 60000, 4, 6),
(3, 40000, 4, 7);
```

```
343
344  SELECT * FROM Tournament;
```

00 % No issues found Ln:

Results Messages

	TournamentID	TournamentName	StartDate	EndDate	PrizePool	Status	OrganizerID	GameID
1	1	PMGC Nepal Series 2026	2026-01-10	2026-01-20	500000.00	Completed	1	1
2	2	MLBB Champions Cup	2026-02-01	2026-02-10	300000.00	Ongoing	2	2
3	3	Valorant Masters Kathmandu	2026-03-05	2026-03-15	700000.00	Upcoming	3	3
4	4	Free Fire Fury League	2026-01-25	2026-01-30	200000.00	Completed	1	4
5	5	Dota 2 Himalayan Cup	2026-04-01	2026-04-12	450000.00	Upcoming	4	5

```
344  SELECT * FROM Match_Result;
```

100 % No issues found

Results Messages

	ResultID	Placement	KillPoints	PlacementPoints	TotalPoints	MatchID	TeamID
1	3	1	10	10	20	1	1
2	4	2	6	8	14	1	2
3	5	3	4	6	10	1	3
4	6	4	2	4	6	1	4
5	7	1	12	10	22	2	2
6	8	2	8	8	16	2	1
7	9	3	5	6	11	2	4
8	10	4	3	4	7	2	3
9	11	1	9	10	19	3	1
10	12	2	7	8	15	3	2
11	13	1	11	10	21	4	1
12	14	2	9	8	17	4	2
13	15	1	5	10	15	5	3
14	16	2	3	8	11	5	4
15	17	1	4	10	14	6	6
16	18	2	2	8	10	6	7
17	19	1	6	10	16	7	3
18	20	2	3	8	11	7	6
19	21	1	7	10	17	8	3
20	22	2	4	8	12	8	7
21	23	1	10	10	20	9	5
22	24	2	7	8	15	9	6
23	25	3	4	6	10	9	7
24	26	4	2	4	6	9	8
25	27	1	9	10	19	10	6

Query executed successfully. DESKTOP-L475BA8\SQLEXPRESS0...

Fig 3.3:Sample Data Table

CHAPTER 4: SQL IMPLEMENTATION

This chapter demonstrates the SQL Server queries and database objects used to operate on the Esports Tournament Tracker and Management System. All queries are written in Microsoft SQL Server (T-SQL) syntax and are based on the schema and sample data introduced in Chapter 3.

4.1 INSERT Queries

The bulk sample data was loaded in Section 3.4. The following additional INSERT statements illustrate how new records are added to the system during normal operation.

```
-- Add a new team
INSERT INTO Team (TeamName, CaptainName, Region, CoachName)
VALUES ('Biratnagar Blazers', 'Nischal Karki', 'Biratnagar', 'Uttam Basnet');

-- Add a new player for that team (assume the new TeamID is 9)
INSERT INTO Player (PlayerName, IGN, Role, Country, TeamID)
VALUES ('Nischal Karki', 'NischalB', 'IGL', 'Nepal', 9);

-- Register the new team for the upcoming Valorant tournament
INSERT INTO Registration (Status, TournamentID, TeamID)
VALUES ('Pending', 3, 9);
```

Because RegistrationDate and Status have DEFAULT constraints, the RegistrationDate column may be omitted and SQL Server will automatically use the current date; Status was supplied explicitly here to demonstrate the CHECK constraint.

4.2 UPDATE Queries

UPDATE statements are used to modify existing records. Examples for each core entity are shown below.

Updating team information (changing a coach):

```
UPDATE Team
SET CoachName = 'Rabin Khadka'
WHERE TeamName = 'Everest Esports';
```

Updating tournament status (marking a tournament as completed):

```
UPDATE Tournament
SET Status = 'Completed'
WHERE TournamentID = 2;
```

Updating match information (rescheduling a match date):

```
UPDATE Match
SET MatchDate = '2026-03-08'
WHERE MatchID = 12;
```

Updating player information (a player transferring to a new team):

```
UPDATE Player
SET TeamID = 7
WHERE IGN = 'YubrajY';
```

4.3 DELETE Queries

DELETE statements must respect foreign-key relationships: a row cannot be deleted while it is still referenced by a row in another table. The following examples show safe deletions.

Deleting a registration that was never confirmed:

```
DELETE FROM Registration
WHERE TournamentID = 2 AND TeamID = 7 AND Status = 'Pending';
```

Deleting a sponsor record that was entered in error:

```
DELETE FROM Sponsor
WHERE SponsorName = 'Himalayan Java Coffee' AND TournamentID = 4;
```

Attempting to delete a team that still has dependent Player, Registration, Match_Result, or Prize rows will fail with a foreign-key constraint error, which is the expected and desired behaviour, since it prevents orphaned records:

```
-- Match_Result rows:
DELETE FROM Team WHERE TeamID = 1;
```

To remove a team correctly, its dependent rows must be deleted first (or the team should simply be retained with historical data intact, which is the recommended practice for preserving tournament history):

```
DELETE FROM Player WHERE TeamID = 9;
DELETE FROM Registration WHERE TeamID = 9;
DELETE FROM Team WHERE TeamID = 9;
```

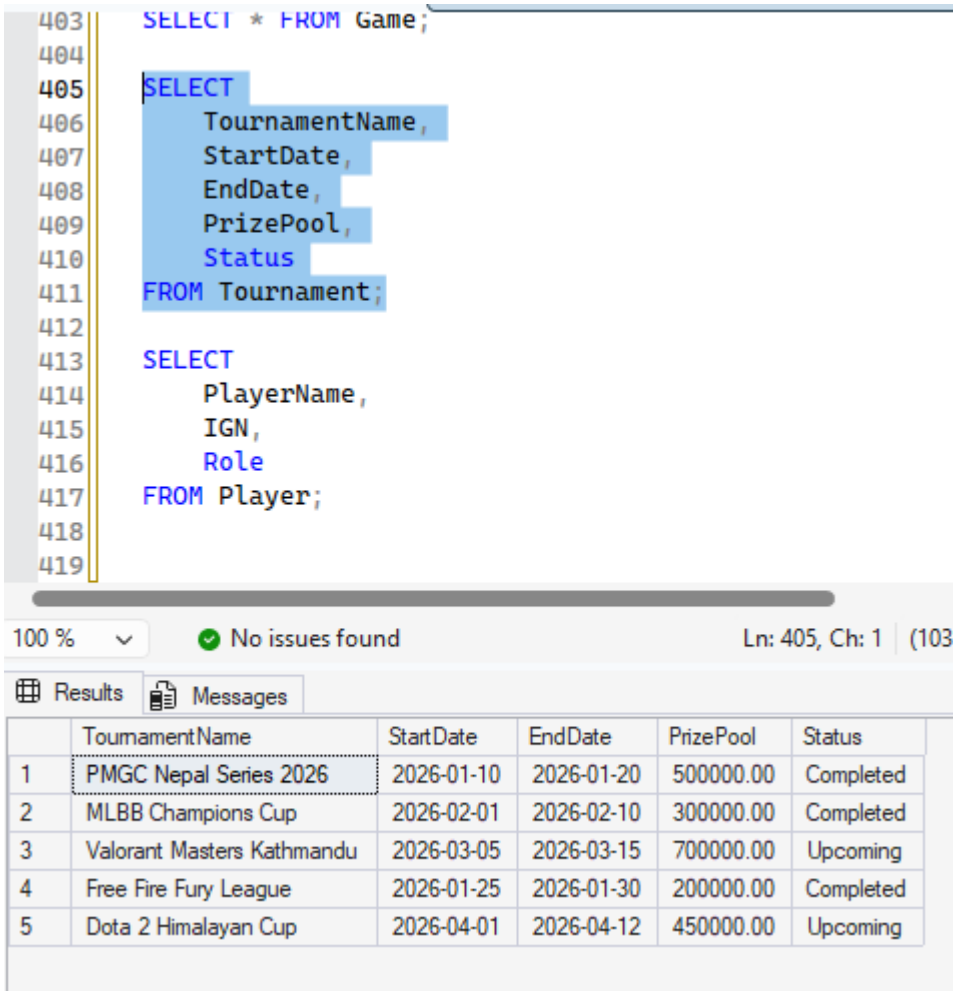
4.4 SELECT Queries

Basic SELECT statements retrieve data from a single table.

```
SELECT * FROM Game;

SELECT TournamentName, StartDate, EndDate, PrizePool, Status FROM Tournament;

SELECT PlayerName, IGN, Role FROM Player;
```



	TournamentName	StartDate	EndDate	PrizePool	Status
1	PMGC Nepal Series 2026	2026-01-10	2026-01-20	500000.00	Completed
2	MLBB Champions Cup	2026-02-01	2026-02-10	300000.00	Completed
3	Valorant Masters Kathmandu	2026-03-05	2026-03-15	700000.00	Upcoming
4	Free Fire Fury League	2026-01-25	2026-01-30	200000.00	Completed
5	Dota 2 Himalayan Cup	2026-04-01	2026-04-12	450000.00	Upcoming

Fig 4.1:Select Querry

4.5 WHERE

The WHERE clause filters rows based on a condition.

```
-- Tournaments that are currently ongoing
SELECT TournamentName, Status FROM Tournament
WHERE Status = 'Ongoing';

-- Tournaments with a prize pool greater than 300,000
SELECT TournamentName, PrizePool FROM Tournament
WHERE PrizePool > 300000;

-- Players who play the 'Fragger' role from Nepal
SELECT PlayerName, IGN FROM Player
WHERE Role = 'Fragger' AND Country = 'Nepal';
```

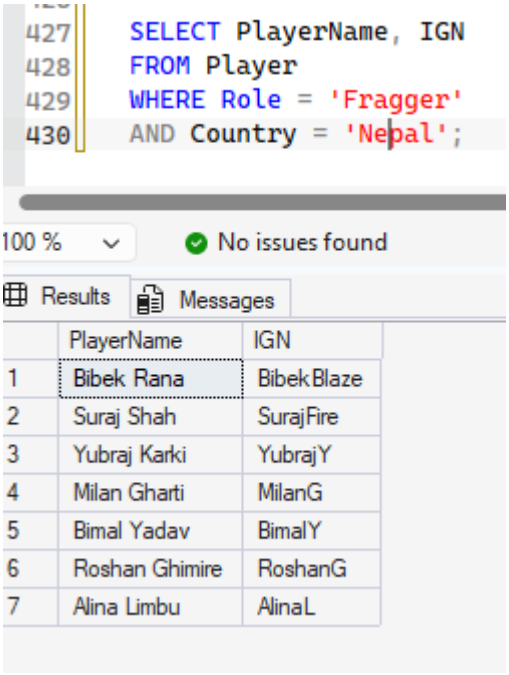


Fig 4.2:Where Querry

4.6 ORDER BY

ORDER BY sorts the result set by one or more columns, ascending (ASC, the default) or descending (DESC).

Sorting tournaments by start date:

```
SELECT TournamentName, StartDate FROM Tournament ORDER BY StartDate ASC;
```

Sorting tournaments by prize pool (highest first):

```
SELECT TournamentName, PrizePool FROM Tournament ORDER BY PrizePool DESC;
```

Sorting players alphabetically by name:

```
SELECT PlayerName, IGN FROM Player ORDER BY PlayerName ASC;
```

Sorting teams by their total points (requires aggregation across Match_Result, discussed further in Section 4.8):

```
SELECT T.TeamName, SUM(MR.TotalPoints) AS TeamPoints
FROM Team T
JOIN Match_Result MR ON T.TeamID = MR.TeamID
GROUP BY T.TeamName
ORDER BY TeamPoints DESC;
```

4.7 Aggregate Functions

Aggregate functions summarize data across multiple rows.

```
-- COUNT: total number of registered teams
SELECT COUNT(*) AS TotalRegistrations FROM Registration;

-- SUM: total prize money awarded across all tournaments
SELECT SUM(PrizeAmount) AS TotalPrizeAwarded FROM Prize;

-- AVG: average prize pool across all tournaments
SELECT AVG(PrizePool) AS AveragePrizePool FROM Tournament;

-- MAX: highest kill points recorded in a single match
SELECT MAX(KillPoints) AS HighestKillPoints FROM Match_Result;

-- MIN: earliest tournament start date
SELECT MIN(StartDate) AS EarliestTournament FROM Tournament;
```

4.8 GROUP BY

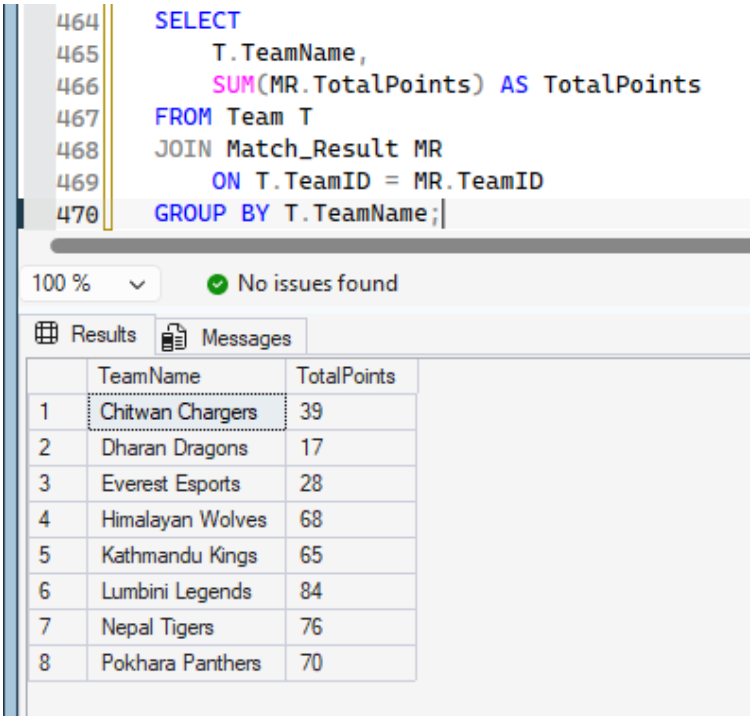
GROUP BY groups rows that share a common value so that aggregate functions can be applied to each group separately.

Total points earned by each team:

```
SELECT T.TeamName, SUM(MR.TotalPoints) AS TotalPoints
FROM Team T
JOIN Match_Result MR ON T.TeamID = MR.TeamID
GROUP BY T.TeamName;
```

Number of teams registered per tournament:

```
SELECT TO2.TournamentName, COUNT(R.TeamID) AS NumberOfTeams
FROM Tournament TO2
JOIN Registration R ON TO2.TournamentID = R.TournamentID
GROUP BY TO2.TournamentName;
```



	TeamName	TotalPoints
1	Chitwan Chargers	39
2	Dharan Dragons	17
3	Everest Esports	28
4	Himalayan Wolves	68
5	Kathmandu Kings	65
6	Lumbini Legends	84
7	Nepal Tigers	76
8	Pokhara Panthers	70

Fig 4.3:GROUP BY Querry

Number of players per team:

```
SELECT T.TeamName, COUNT(P.PlayerID) AS NumberOfPlayers
FROM Team T
JOIN Player P ON T.TeamID = P.TeamID
GROUP BY T.TeamName;
```

Total prize amount per tournament:

```
SELECT TO2.TournamentName, SUM(PZ.PrizeAmount) AS TotalPrizeAmount
FROM Tournament TO2
JOIN Prize PZ ON TO2.TournamentID = PZ.TournamentID
GROUP BY TO2.TournamentName;
```

4.9 HAVING

HAVING filters groups produced by GROUP BY, unlike WHERE which filters individual rows before grouping.

```
-- Teams whose total points across all matches exceed 50
SELECT T.TeamName, SUM(MR.TotalPoints) AS TotalPoints
```

```
FROM Team T
JOIN Match_Result MR ON T.TeamID = MR.TeamID
GROUP BY T.TeamName
HAVING SUM(MR.TotalPoints) > 50
ORDER BY TotalPoints DESC;
```

This query first groups match results by team, computes each team's total points, and then keeps only the groups (teams) whose total exceeds 50 points.

4.10 JOIN Queries

JOIN operations combine rows from two or more tables based on a related column.

INNER JOIN -Tournament with Organizer

```
SELECT T.TournamentName, O.Name AS OrganizerName, O.Organization
FROM Tournament T
INNER JOIN Organizer O ON T.OrganizerID = O.OrganizerID;
```

INNER JOIN -Tournament with Game

```
SELECT T.TournamentName, G.GameName, G.Genre
FROM Tournament T
INNER JOIN Game G ON T.GameID = G.GameID;
```

INNER JOIN -Teams registered in a specific tournament

```
SELECT TE.TeamName, R.Status, R.RegistrationDate
FROM Registration R
INNER JOIN Team TE ON R.TeamID = TE.TeamID
WHERE R.TournamentID = 1;
```

INNER JOIN -Players with their teams

```
SELECT P.PlayerName, P.IGN, TE.TeamName
FROM Player P
INNER JOIN Team TE ON P.TeamID = TE.TeamID;
```

LEFT JOIN -All tournaments, including those without sponsors yet

```
SELECT T.TournamentName, S.SponsorName, S.Amount
FROM Tournament T
LEFT JOIN Sponsor S ON T.TournamentID = S.TournamentID;
```

The LEFT JOIN ensures every tournament appears in the result even if it currently has no sponsor rows (for example, an upcoming tournament); unmatched sponsor columns are returned as NULL.

```
506    SELECT
507        T.TournamentName,
508        O.Name AS OrganizerName,
509        O.Organization
510    FROM Tournament T
511    INNER JOIN Organizer O
512        ON T.OrganizerID = O.OrganizerID;
```

100 % No issues found

Results Messages

	TournamentName	OrganizerName	Organization
1	PMGC Nepal Series 2026	Rajesh Sharma	ProGaming Nepal
2	MLBB Champions Cup	Anita Gurung	LevelUp Events
3	Valorant Masters Kathmandu	Bishal Karki	Arena Esports
4	Free Fire Fury League	Rajesh Sharma	ProGaming Nepal
5	Dota 2 Himalayan Cup	Sunita Rai	Champions League Esports

Fig 4.4:JOIN Querry

Multiple-table JOIN -Match results with team and match information

```
SELECT TN.TournamentName, M.MatchNumber, M.Stage, TE.TeamName,
       MR.Placement, MR.KillPoints, MR.PlacementPoints, MR.TotalPoints
FROM Match_Result MR
JOIN Match M ON MR.MatchID = M.MatchID
JOIN Tournament TN ON M.TournamentID = TN.TournamentID
JOIN Team TE ON MR.TeamID = TE.TeamID
ORDER BY TN.TournamentName, M.MatchNumber, MR.Placement;
```

4.11 Nested Queries

Nested queries (subqueries) are queries placed inside another query, often used to compute a value that is then used in the outer query's condition.

Team with the highest total points

```
SELECT T.TeamName, SUM(MR.TotalPoints) AS TotalPoints
FROM Team T
JOIN Match_Result MR ON T.TeamID = MR.TeamID
```

```
GROUP BY T.TeamName
HAVING SUM(MR.TotalPoints) = (
    SELECT MAX(TeamTotal.TotalPoints)
    FROM (
        SELECT SUM(TotalPoints) AS TotalPoints
        FROM Match_Result
        GROUP BY TeamID
    ) AS TeamTotal
);
```

Tournament with the highest prize pool

```
SELECT TournamentName, PrizePool
FROM Tournament
WHERE PrizePool = (SELECT MAX(PrizePool) FROM Tournament);
```

Players belonging to teams that participated in a particular tournament

```
SELECT P.PlayerName, P.IGN, P.TeamID
FROM Player P
WHERE P.TeamID IN (
    SELECT R.TeamID
    FROM Registration R
    WHERE R.TournamentID = 1
);
```

4.12 Dynamic Leaderboard Query

As explained in Section 2.7, no separate Leaderboard table is stored in this database. Instead, the leaderboard for any tournament is calculated dynamically at query time by aggregating the Match_Result rows for the matches belonging to that tournament.

```
-- Dynamic leaderboard for Tournament 1 (PMGC Nepal Series 2026)
SELECT TE.TeamName,
       SUM(MR.KillPoints)      AS TotalKillPoints,
       SUM(MR.PlacementPoints) AS TotalPlacementPoints,
       SUM(MR.TotalPoints)     AS TotalPoints,
       COUNT(MR.MatchID)       AS MatchesPlayed
FROM Match_Result MR
JOIN Match M   ON MR.MatchID = M.MatchID
JOIN Team TE   ON MR.TeamID = TE.TeamID
WHERE M.TournamentID = 1
```

```
GROUP BY TE.TeamName
ORDER BY TotalPoints DESC;
```

```
605    SELECT
606        TE.TeamName,
607        SUM(MR.KillPoints) AS TotalKillPoints,
608        SUM(MR.PlacementPoints) AS TotalPlacementPoints,
609        SUM(MR.TotalPoints) AS TotalPoints,
610        COUNT(MR.MatchID) AS MatchesPlayed
611    FROM Match_Result MR
612    JOIN Match M
613        ON MR.MatchID = M.MatchID
614    JOIN Team TE
615        ON MR.TeamID = TE.TeamID
616    WHERE M.TournamentID = 1
617    GROUP BY TE.TeamName
618    ORDER BY TotalPoints DESC;
619
620
```

100 % ✓ No issues found Ln: 618,

Results Messages

	TeamName	TotalKillPoints	TotalPlacementPoints	TotalPoints	MatchesPlayed
1	Nepal Tigers	38	38	76	4
2	Himalayan Wolves	34	34	68	4
3	Kathmandu Kings	7	10	17	2
4	Everest Esports	7	10	17	2

Fig 4.5: Dynamic Leaderboard Query

Why the leaderboard is calculated dynamically rather than stored:

- Storing a leaderboard as a separate table would duplicate data that is already derivable from Match_Result, violating 3NF (see Section 2.7) and risking transitive dependency.
- A stored leaderboard would need to be updated every time a match result changes, which introduces the risk of the leaderboard becoming out of sync with the underlying results (an update anomaly).
- Calculating the leaderboard using SUM(), GROUP BY, and ORDER BY guarantees that it always reflects the current, correct state of the Match_Result table.

- This approach directly demonstrates practical use of aggregate functions, GROUP BY, JOIN, and ORDER BY, which are core SQL Server DBMS concepts covered by this project.

4.13 Views

Views are stored SELECT queries that can be reused like virtual tables. Three views were created to simplify frequently used, multi-table queries.

1. TournamentSummaryView

Provides a consolidated summary of each tournament, including its organizer and game.

```
CREATE VIEW TournamentSummaryView AS
SELECT T.TournamentID, T.TournamentName, T.StartDate, T.EndDate,
       T.PrizePool, T.Status, O.Name AS OrganizerName, G.GameName
FROM Tournament T
JOIN Organizer O ON T.OrganizerID = O.OrganizerID
JOIN Game G ON T.GameID = G.GameID;

SELECT * FROM TournamentSummaryView ORDER BY StartDate;
```

2. TeamLeaderboardView

Provides a global (all-tournament) leaderboard summarizing each team's total points across every match it has played.

```
CREATE VIEW TeamLeaderboardView AS
SELECT TE.TeamID, TE.TeamName,
       SUM(MR.KillPoints)      AS TotalKillPoints,
       SUM(MR.PlacementPoints) AS TotalPlacementPoints,
       SUM(MR.TotalPoints)     AS TotalPoints,
       COUNT(MR.MatchID)       AS MatchesPlayed
FROM Team TE
JOIN Match_Result MR ON TE.TeamID = MR.TeamID
GROUP BY TE.TeamID, TE.TeamName;

SELECT * FROM TeamLeaderboardView ORDER BY TotalPoints DESC;
```

3. MatchResultsView

Provides a readable, joined view of every match result along with the tournament, match, and team it belongs to.

```
CREATE VIEW MatchResultsView AS
SELECT TN.TournamentName, M.MatchNumber, M.Stage, M.MapName,
       TE.TeamName, MR.Placement, MR.KillPoints,
       MR.PlacementPoints, MR.TotalPoints
FROM Match_Result MR
JOIN Match M ON MR.MatchID = M.MatchID
JOIN Tournament TN ON M.TournamentID = TN.TournamentID
JOIN Team TE ON MR.TeamID = TE.TeamID;

SELECT * FROM MatchResultsView WHERE TournamentName = 'PMGC Nepal Series 2026';
```

```
636  SELECT *
637  FROM TournamentSummaryView
638  ORDER BY StartDate;
639
```

100 % 3 0

Results Messages

	TournamentID	TournamentName	StartDate	EndDate	PrizePool	Status	OrganizerName	GameName
1	1	PMGC Nepal Series 2026	2026-01-10	2026-01-20	500000.00	Completed	Rajesh Sharma	PUBG Mobile
2	4	Free Fire Fury League	2026-01-25	2026-01-30	200000.00	Completed	Rajesh Sharma	Free Fire
3	2	MLBB Champions Cup	2026-02-01	2026-02-10	300000.00	Completed	Anita Gurung	Mobile Legends: Bang Bang
4	3	Valorant Masters Kathmandu	2026-03-05	2026-03-15	700000.00	Upcoming	Bishal Karki	Valorant
5	5	Dota 2 Himalayan Cup	2026-04-01	2026-04-12	450000.00	Upcoming	Sunita Rai	Dota 2

```
655  SELECT *
656  FROM TeamLeaderboardView
657  ORDER BY TotalPoints DESC;
```

00 % 3 0

Results Messages

	TeamID	TeamName	TotalKillPoints	TotalPlacementPoints	TotalPoints	MatchesPlayed
1	6	Lumbini Legends	32	52	84	6
2	1	Nepal Tigers	38	38	76	4
3	5	Pokhara Panthers	32	38	70	4
4	2	Himalayan Wolves	34	34	68	4
5	3	Kathmandu Kings	25	40	65	5
6	7	Chitwan Chargers	13	26	39	4
7	4	Everest Esports	10	18	28	3
8	8	Dharan Dragons	7	10	17	2

```
678  SELECT *
679  FROM MatchResultsView
680  WHERE TournamentName = 'PMGC Nepal Series 2026';
```

100 % ⊗ 3 ⚠ 0

Results | Messages

	TournamentName	MatchNumber	Stage	MapName	TeamName	Placement	KillPoints	PlacementPoints	TotalPoints
1	PMGC Nepal Series 2026	1	Group Stage	Erangel	Nepal Tigers	1	10	10	20
2	PMGC Nepal Series 2026	1	Group Stage	Erangel	Himalayan Wolves	2	6	8	14
3	PMGC Nepal Series 2026	1	Group Stage	Erangel	Kathmandu Kings	3	4	6	10
4	PMGC Nepal Series 2026	1	Group Stage	Erangel	Everest Esports	4	2	4	6
5	PMGC Nepal Series 2026	2	Group Stage	Miramar	Nepal Tigers	2	8	8	16
6	PMGC Nepal Series 2026	2	Group Stage	Miramar	Himalayan Wolves	1	12	10	22
7	PMGC Nepal Series 2026	2	Group Stage	Miramar	Kathmandu Kings	4	3	4	7
8	PMGC Nepal Series 2026	2	Group Stage	Miramar	Everest Esports	3	5	6	11
9	PMGC Nepal Series 2026	3	Semifinal	Sanhok	Nepal Tigers	1	9	10	19
10	PMGC Nepal Series 2026	3	Semifinal	Sanhok	Himalayan Wolves	2	7	8	15
11	PMGC Nepal Series 2026	4	Final	Erangel	Nepal Tigers	1	11	10	21
12	PMGC Nepal Series 2026	4	Final	Erangel	Himalayan Wolves	2	9	8	17

Fig 4.6: View Query

4.14 Stored Procedures

Stored procedures encapsulate common operations into reusable, parameterized SQL Server routines.

1. RegisterTeamForTournament

Registers a team for a tournament, preventing duplicate registrations.

```
CREATE PROCEDURE RegisterTeamForTournament
    @TournamentID INT,
    @TeamID       INT,
    @Status       VARCHAR(20) = 'Pending'
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM Registration
               WHERE TournamentID = @TournamentID AND TeamID = @TeamID)
    BEGIN
        RAISERROR('This team is already registered for the tournament.', 16, 1);
        RETURN;
    END

    INSERT INTO Registration (RegistrationDate, Status, TournamentID, TeamID)
```

```
    VALUES (GETDATE(), @Status, @TournamentID, @TeamID);
END;

EXEC RegisterTeamForTournament @TournamentID = 5, @TeamID = 3, @Status = 'Pending';
```

2. AddMatchResult

Inserts a match result for a team, after checking that the team is registered for the match's tournament.

```
CREATE PROCEDURE AddMatchResult
    @MatchID          INT,
    @TeamID            INT,
    @Placement         INT,
    @KillPoints        INT,
    @PlacementPoints   INT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @TournamentID INT;
    SELECT @TournamentID = TournamentID FROM Match WHERE MatchID = @MatchID;

    IF NOT EXISTS (SELECT 1 FROM Registration
                   WHERE TournamentID = @TournamentID AND TeamID = @TeamID)
    BEGIN
        RAISERROR('The team is not registered for this tournament.', 16, 1);
        RETURN;
    END

    INSERT INTO Match_Result (Placement, KillPoints, PlacementPoints, MatchID,
TeamID)
    VALUES (@Placement, @KillPoints, @PlacementPoints, @MatchID, @TeamID);
END;

EXEC AddMatchResult @MatchID = 12, @TeamID = 3, @Placement = 3, @KillPoints = 4,
@PlacementPoints = 6;
```

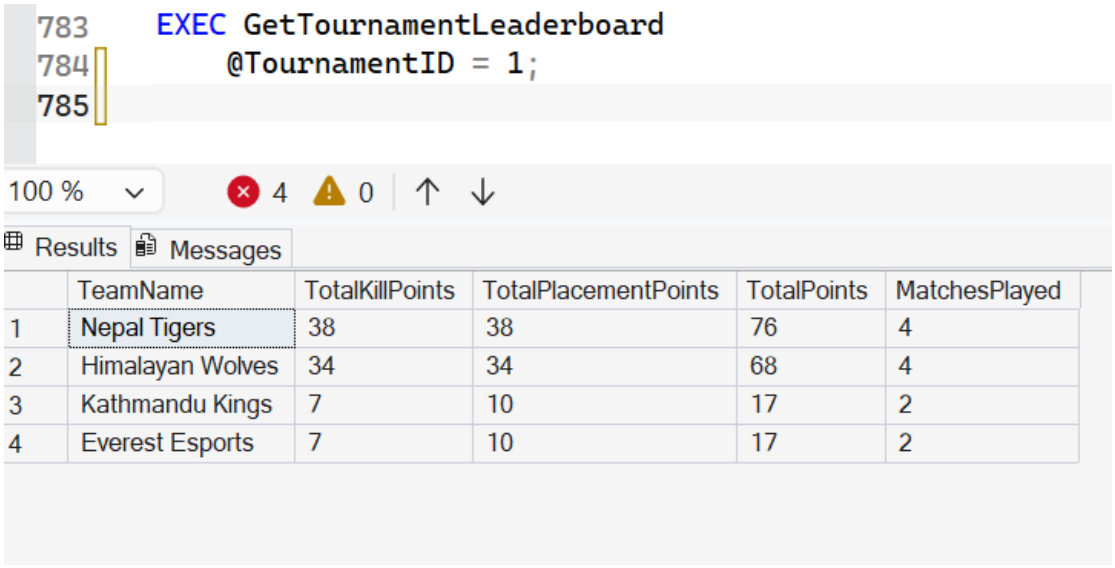
3. GetTournamentLeaderboard

Returns the dynamic leaderboard for a given tournament, reusing the aggregation logic from Section 4.12.

```
CREATE PROCEDURE GetTournamentLeaderboard
    @TournamentID INT
AS
```

```
BEGIN
    SET NOCOUNT ON;
    SELECT TE.TeamName,
           SUM(MR.KillPoints)      AS TotalKillPoints,
           SUM(MR.PlacementPoints) AS TotalPlacementPoints,
           SUM(MR.TotalPoints)     AS TotalPoints,
           COUNT(MR.MatchID)       AS MatchesPlayed
    FROM Match_Result MR
    JOIN Match M ON MR.MatchID = M.MatchID
    JOIN Team TE ON MR.TeamID = TE.TeamID
    WHERE M.TournamentID = @TournamentID
    GROUP BY TE.TeamName
    ORDER BY TotalPoints DESC;
END;

EXEC GetTournamentLeaderboard @TournamentID = 1;
```



	TeamName	TotalKillPoints	TotalPlacementPoints	TotalPoints	MatchesPlayed
1	Nepal Tigers	38	38	76	4
2	Himalayan Wolves	34	34	68	4
3	Kathmandu Kings	7	10	17	2
4	Everest Esports	7	10	17	2

Fig 4.7: GetTournamentLeaderboard execution

4.15 Transactions

A transaction groups one or more SQL statements so that they are executed as a single logical unit of work: either all statements succeed and are permanently saved (COMMIT), or if an error occurs, all changes made so far are undone (ROLLBACK).

This is essential in the Esports Tournament Tracker and Management System because several operations, such as registering a team or recording a match result, may involve inserting into more than one table or must satisfy business rules that a simple INSERT cannot enforce alone.

Transaction 1: Register a team into a tournament

```
BEGIN TRANSACTION;

BEGIN TRY
    IF EXISTS (SELECT 1 FROM Registration
               WHERE TournamentID = 3 AND TeamID = 4)
    BEGIN
        RAISERROR('Team is already registered for this tournament.', 16, 1);
    END

    INSERT INTO Registration (RegistrationDate, Status, TournamentID, TeamID)
    VALUES (GETDATE(), 'Pending', 3, 4);

    COMMIT TRANSACTION;
    PRINT 'Registration completed successfully.';
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    PRINT 'Registration failed: ' + ERROR_MESSAGE();
END CATCH;
```

Transaction 2: Insert a match result safely

```
BEGIN TRANSACTION;

BEGIN TRY
    DECLARE @MatchID INT = 8, @TeamID INT = 6;

    IF NOT EXISTS (SELECT 1 FROM Match WHERE MatchID = @MatchID)
    BEGIN
        RAISERROR('Invalid MatchID.', 16, 1);
    END

    IF NOT EXISTS (SELECT 1 FROM Team WHERE TeamID = @TeamID)
    BEGIN
        RAISERROR('Invalid TeamID.', 16, 1);
    END
```

```
    INSERT INTO Match_Result (Placement, KillPoints, PlacementPoints, MatchID,
TeamID)
    VALUES (3, 2, 6, @MatchID, @TeamID);

    COMMIT TRANSACTION;
    PRINT 'Match result recorded successfully.';
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    PRINT 'Match result insertion failed: ' + ERROR_MESSAGE();
END CATCH;
```

Transactions are important in this system because they guarantee data consistency: for example, if a match result insertion were interrupted midway (due to a constraint violation, a hardware failure, or a network error), the TRY...CATCH block combined with ROLLBACK TRANSACTION ensures that no partial or inconsistent data is left in the database. This is particularly important for Match_Result, since an incomplete or duplicated result would directly corrupt the dynamically calculated leaderboard shown in Section 4.12.

```
878        (RegistrationDate, Status, TournamentID, TeamID)
879        VALUES
880        (GETDATE(), 'Confirmed', 5, 8);
881
882        COMMIT TRANSACTION;
883
884        PRINT 'Registration completed successfully. Transaction COMMITTED.';
885
886    END TRY
887
888    BEGIN CATCH
889
890        ROLLBACK TRANSACTION;
891
892        PRINT 'Registration failed: ' + ERROR_MESSAGE();
893
894    END CATCH;
```

110 % 4 0

Messages

```
(1 row affected)
Registration completed successfully. Transaction COMMITTED.

Completion time: 2026-08-15T00:07:28.7115795+05:45
```

Fig 4.8: Successful transaction / COMMIT



Fig 4.9: Failed transaction / ROLLBACK

CHAPTER 5: TESTING AND RESULTS

5.1 Testing

The database was tested in SSMS to verify that constraints, queries, views, stored procedures, and transactions behave as designed. The key test cases are summarized below.

Test Case	Input / Action	Expected Result	Actual Result	Status
Valid team registration	Register an unregistered team for an open tournament	Row inserted into Registration	Row inserted successfully	Pass
Duplicate registration	Register the same team for the same tournament twice	UNIQUE constraint violation / procedure raises error	Insert rejected with error	Pass
Invalid points	Insert Match_Result with KillPoints = -5	CHECK constraint violation	Insert rejected with error	Pass
Invalid tournament dates	Insert Tournament with EndDate earlier than StartDate	CHECK constraint violation	Insert rejected with error	Pass
Match result insertion	Insert a valid Match_Result row via AddMatchResult	Row inserted, TotalPoints computed automatically	Row inserted; TotalPoints correct	Pass
Leaderboard generation	Run the dynamic leaderboard query for Tournament 1	Teams ranked correctly by TotalPoints	Correct ranking returned	Pass

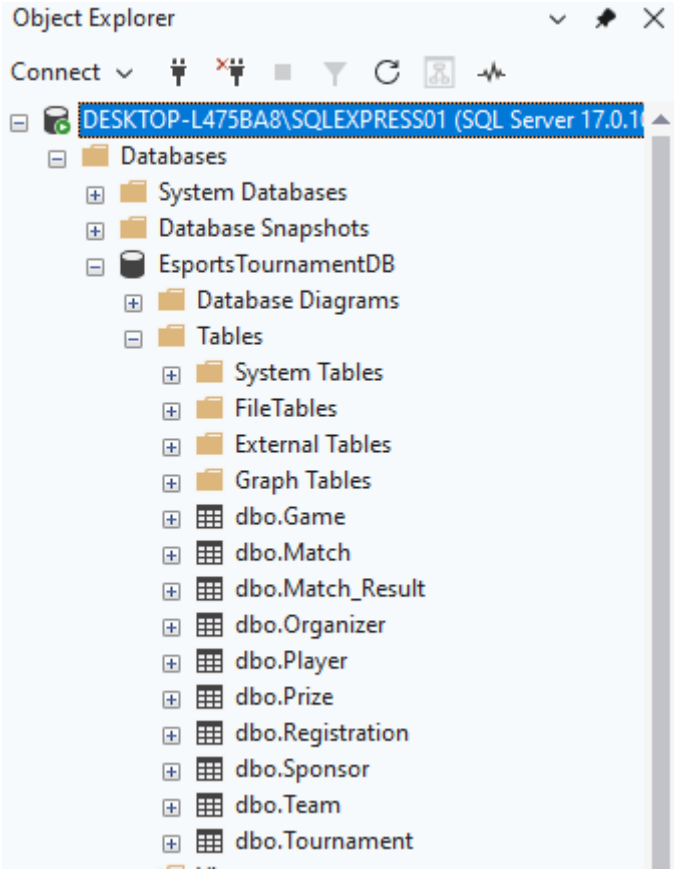
Test Case	Input / Action	Expected Result	Actual Result	Status
View execution	SELECT * FROM TeamLeaderboardView	Aggregated results returned without rewriting the join/group logic	Correct results returned	Pass
Stored procedure execution	EXEC GetTournamentLeaderboard @TournamentID = 1	Leaderboard for Tournament 1 returned	Correct leaderboard returned	Pass
Transaction success	Run Transaction 1 with a valid, unregistered team	COMMIT executes; row is saved	Row saved and committed	Pass
Transaction rollback	Run Transaction 1 with a team that is already registered	RAISERROR triggers CATCH block; ROLLBACK undoes changes	No row inserted; rollback message printed	Pass

Note: The “Actual Result” and “Status” columns above should be verified and filled in based on the actual execution of these test cases in SSMS on the submitted database.

5.2 Query Outputs and Screenshots

The following SSMS screenshots should be captured and inserted at the indicated locations to provide visual evidence of the working system.

- Database creation (EsportsTournamentDB created and visible in Object Explorer).
- All ten tables listed under the database in Object Explorer.
- Sample data displayed for at least the Tournament and Match_Result tables.
- Output of a basic SELECT query (Section 4.4).
- Output of a JOIN query, e.g. tournament with organizer and game (Section 4.10).



- Output of a GROUP BY query, e.g. total points per team (Section 4.8).
- Output of the dynamic leaderboard query (Section 4.12).
- Output of each of the three views (Section 4.13).
- Execution of each of the three stored procedures (Section 4.14).
- Execution of a successful transaction (COMMIT) and a failed transaction (ROLLBACK) (Section 4.15).

```
783    EXEC GetTournamentLeaderboard
784        @TournamentID = 1;
785
```

100 % ✕ 4 ⚠ 0

Results | Messages

	TeamName	TotalKillPoints	TotalPlacementPoints	TotalPoints	MatchesPlayed
1	Nepal Tigers	38	38	76	4
2	Himalayan Wolves	34	34	68	4
3	Kathmandu Kings	7	10	17	2
4	Everest Esports	7	10	17	2

Figure 5.1: Tournament table with sample records

Figure 5.2: Dynamic leaderboard query output for Tournament 1

Figure 5.3: Stored procedure GetTournamentLeaderboard execution

```
605    SELECT
606        TE.TeamName,
607        SUM(MR.KillPoints) AS TotalKillPoints,
608        SUM(MR.PlacementPoints) AS TotalPlacementPoints,
609        SUM(MR.TotalPoints) AS TotalPoints,
610        COUNT(MR.MatchID) AS MatchesPlayed
611    FROM Match_Result MR
612    JOIN Match M
613        ON MR.MatchID = M.MatchID
614    JOIN Team TE
615        ON MR.TeamID = TE.TeamID
616    WHERE M.TournamentID = 1
617    GROUP BY TE.TeamName
618    ORDER BY TotalPoints DESC;
619
620
```

100 % ✓ No issues found Ln: 618,

Results | Messages

	TeamName	TotalKillPoints	TotalPlacementPoints	TotalPoints	MatchesPlayed
1	Nepal Tigers	38	38	76	4
2	Himalayan Wolves	34	34	68	4
3	Kathmandu Kings	7	10	17	2
4	Everest Esports	7	10	17	2

Figure 5.4: Transaction rollback on duplicate registration

5.3 Observations

The following observations were made while testing the Esports Tournament Tracker and Management System:

- Data integrity was maintained throughout testing: primary key and foreign key constraints prevented orphaned or duplicate records.
- Foreign-key relationships worked correctly, correctly linking Tournament to Organizer and Game, Registration to Tournament and Team, Match_Result to Match and Team, and Prize to Tournament and Team.



- CHECK, UNIQUE, and NOT NULL constraints successfully prevented invalid data, such as negative points, invalid tournament dates, and duplicate registrations, from being inserted.
- SELECT, WHERE, JOIN, GROUP BY, and HAVING queries successfully retrieved the required tournament, team, player, and match information.
- Views (TournamentSummaryView, TeamLeaderboardView, MatchResultsView) simplified repeated multi-table queries by allowing them to be reused as simple SELECT statements.

- Stored procedures (RegisterTeamForTournament, AddMatchResult, GetTournamentLeaderboard) automated common operations and enforced additional business rules beyond what table constraints alone could express.
- Transactions with TRY...CATCH and ROLLBACK maintained consistency by ensuring that invalid or duplicate operations did not leave partial data in the database.

CHAPTER 6: CONCLUSION

6.1 Summary

This project designed and implemented the Esports Tournament Tracker and Management System, a relational database for managing esports tournaments, games, organizers, teams, players, registrations, matches, match results, sponsors, and prizes, using Microsoft SQL Server and SQL Server Management Studio (SSMS).

The database consists of ten core entities -Organizer, Game, Tournament, Team, Player, Registration, Match, Match_Result, Sponsor, and Prize -connected through primary and foreign keys. The Registration entity resolves the many-to-many relationship between Tournament and Team, and the Match_Result entity resolves the many-to-many relationship between Match and Team. The schema was normalized up to the Third Normal Form (3NF), and, in keeping with 3NF principles, no separate Leaderboard table was created; instead, team rankings are calculated dynamically from Match_Result using SQL aggregate functions.

The implementation demonstrated a broad range of SQL Server DBMS concepts, including appropriate data types, NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY, and FOREIGN KEY constraints; INSERT, UPDATE, DELETE, and SELECT operations; filtering with WHERE; sorting with ORDER BY; aggregate functions (COUNT, SUM, AVG, MAX, MIN); grouping and filtering groups with GROUP BY and HAVING; INNER and LEFT JOIN operations across multiple tables; nested (sub)queries; views; stored procedures; and transactions using BEGIN TRANSACTION, COMMIT, ROLLBACK, and TRY...CATCH.

Overall, the system successfully satisfies the requirements of the DBMS course project by demonstrating a complete database design and implementation cycle, from requirement analysis and ER modelling through to normalization, SQL Server implementation, and testing, applied to a realistic and relevant esports tournament management scenario.

6.2 Future Improvements

While the current project focuses on database design and SQL Server implementation, the system could be extended in the following ways in future work:

- Developing a web-based interface for organizers, teams, and spectators to interact with the system.
- Developing a mobile application for real-time tournament updates.
- Enabling online team registration directly through a public-facing form.
- Automating tournament bracket generation based on registered teams.
- Adding live match tracking and score updates during ongoing matches.
- Building a player statistics dashboard showing historical performance across tournaments.
- Adding automated notifications (email/SMS) for match schedules and results.
- Integrating a payment gateway for sponsorship payments and prize disbursement.
- Adding authentication and role-based access control (e.g., organizer, team captain, spectator roles).
- Providing a real-time leaderboard dashboard using live queries or a caching layer.
- Integrating with external esports platforms and APIs for automatic match and player data synchronization.

Appendix: Mapping of Implemented Features to DBMS Assignment Requirements

The table below maps each DBMS requirement of this assignment to the section of this report where it is demonstrated.

Requirement	Demonstrated In
Entity identification & attributes	Sections 2.2, 2.3
Relationships & cardinality	Section 2.4
ER Diagram	Section 2.5
Relational schema	Section 2.6
Normalization (1NF, 2NF, 3NF)	Section 2.7
Primary keys / Foreign keys	Sections 2.6, 3.2
Appropriate data types	Section 3.2
NOT NULL / UNIQUE / CHECK / DEFAULT constraints	Sections 3.2, 3.3
INSERT operations	Sections 3.4, 4.1
UPDATE operations	Section 4.2
DELETE operations	Section 4.3
SELECT / WHERE / ORDER BY	Sections 4.4, 4.5, 4.6
Aggregate functions	Section 4.7
GROUP BY / HAVING	Sections 4.8, 4.9
JOIN operations	Section 4.10
Nested queries	Section 4.11
Dynamic leaderboard (no stored Leaderboard table)	Section 4.12
Views	Section 4.13

Requirement	Demonstrated In
Stored procedures	Section 4.14
Transactions	Section 4.15
Testing	Chapter 5